

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DE PROCESSADOR RV32I MONOCICLO

Disciplina: Arquitetura de Computadores II

Instituição: Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Curso: Ciência da Computação

INTEGRANTES DO GRUPO:

- Rafael Assis Carvalho Lacerda
- Leonardo Stuart de Almeida Ramalho
- Luca Guimarães Lodi
- Gabriel Assis Carvalho Lacerda

1. INTRODUÇÃO

Este relatório documenta o desenvolvimento, a especificação microarquitetural e a validação funcional de um processador de 32 bits baseado na arquitetura de conjunto de instruções (ISA) RISC-V, especificamente o subconjunto base RV32I. O projeto adota o controle monociclo (single-cycle), no qual toda e qualquer instrução realiza seu ciclo completo de busca, decodificação, execução, acesso à memória e escrita de volta em exatamente um ciclo de clock.

2. MAPEAMENTO E DESCRIÇÃO DOS ARQUIVOS DO PROJETO

A estrutura do projeto foi organizada localmente em diretórios específicos de Engenharia de Hardware, separando os arquivos de descrição do circuito (Design) dos arquivos de estímulo e validação (Testbench). Abaixo encontra-se a especificação do papel de cada arquivo no projeto:

Diretorio: /Design (Módulos de Hardware)

- **top_processor.v / design.sv (Módulo Topo / A Placa-Mãe):** São arquivos idênticos em conteúdo. O arquivo `design.sv` representa o nome mandatório exigido pela ferramenta EDA Playground para compilação do circuito principal, enquanto `top_processor.v` é a nomenclatura padronizada do projeto local. Este módulo centraliza a fiação global da CPU, instancia os 5 estágios de processamento, define a memória ROM de instruções e realiza a lógica combinacional de emenda do estágio de Write-Back e barramentos de salto.
- **instruction_fetch.v (Estágio 1 - IF):** Contém a lógica do Program Counter (PC) e o somador dedicado de avanço sequencial.
- **instruction_decode.v (Estágio 2 - ID):** Abriga o Banco de Registradores de 32 bits e o circuito combinacional do Gerador de Imediatos (Imm-Gen).

- **execute.v (Estágio 3 - EX):** Contém a Unidade Lógica e Aritmética (ULA) principal, o decodificador de controle da ULA (ALU Control) e o bloco lógico Avaliador de Condições de Desvio.
- **data_memory.v (Estágio 4 - MEM):** Implementa a memória RAM estática de dados para armazenamento temporário de variáveis do programa.
- **control_unit.v (Unidade de Controle Principal):** Módulo estritamente combinacional responsável por decodificar o campo *opcode* da instrução e emitir os sinais de controle globais (como *reg_write*, *mem_read*, *branch*, *pc_src*, etc.) que orquestram o comportamento de todos os multiplexadores e portas de escrita do datapath.

Diretório: /testbench (Ambiente de Simulação)

- **testbench.sv (Módulo de Estímulo):** O arquivo de simulação principal do ambiente. Responsável por gerar os pulsos elétricos de clock, gerenciar o sinal de inicialização (Reset), carregar o código de máquina no processador e disparar o arquivo de saída de ondas digitais (*dump.vcd*).
- **assembly.txt (Código-Fonte de Software):** Documento em formato de texto contendo o programa de teste escrito em linguagem Assembly RISC-V legível por humanos.
- **program.hex (Código de Máquina Compilado):** Arquivo de texto puro estruturado em linhas hexadecimais de 32 bits. Representa o binário real gerado a partir do assembly.

3. ENGENHARIA DO DATAPATH E ESTRUTURA MODULAR

O caminho de dados divide-se estruturalmente de acordo com o fluxo clássico da arquitetura de computadores:

Estágio 1: Instruction Fetch (IF)

Responsável por buscar a instrução a ser executada na memória de instruções ROM.

- **Componentes:** Contém o registrador síncrono Program Counter (PC), um somador combinacional dedicado para calcular o próximo endereço sequencial ($PC + 4$) e um multiplexador combinacional.
- **Lógica:** O multiplexador seleciona entre o endereço sequencial ($PC + 4$) ou o endereço de destino de um desvio/salto (*pc_target*), baseado no sinal de controle global *pc_src*.

Estágio 2: Instruction Decode (ID)

Responsável por interpretar os bits da instrução e extrair os operandos.

- **Componentes:** Contém o Banco de Registradores (Register File) de 32 bits com 32 posições de uso geral e a Unidade de Extensão de Imediatos (Imm-Gen).
- **Lógica:** A leitura dos registradores rs1 e rs2 é puramente combinacional. O registrador x0 possui um aterramento lógico rígido, retornando sempre 0. A escrita no registrador de destino rd é síncrona (na borda de subida do clock) e condicionada pelo sinal reg_write. O Imm-Gen estende o sinal de immediatos para 32 bits baseando-se no formato da instrução lida (Formatos I, S, B, U e J).

Estágio 3: Execute (EX)

Onde as operações aritméticas, lógicas ou de cálculo de endereço efetivo ocorrem.

- **Componentes:** Contém a Unidade Lógica e Aritmética (ULA), o multiplexador de seleção do segundo operando da ULA, a Unidade de Controle da ULA (ALU Control) e o Avaliador de Condição de Desvio.
- **Lógica:** O multiplexador seleciona se o operando B da ULA será o dado do registrador rs2 ou o imediato estendido, controlado por alu_src. O ALU Control decodifica os campos funct3, funct7 e o sinal alu_op em uma palavra de controle de 4 bits (alu_ctrl), ditando a operação interna da ULA (soma, subtração, shifts lógicos e aritméticos, AND, OR, XOR e comparações). O Avaliador de Desvios calcula de forma combinacional se a condição de desvio foi atendida (ex: igualdade, menor que, maior ou igual), gerando o sinal branch_taken.

Estágio 4: Memory Access (MEM)

Responsável por interagir com a Memória de Dados RAM.

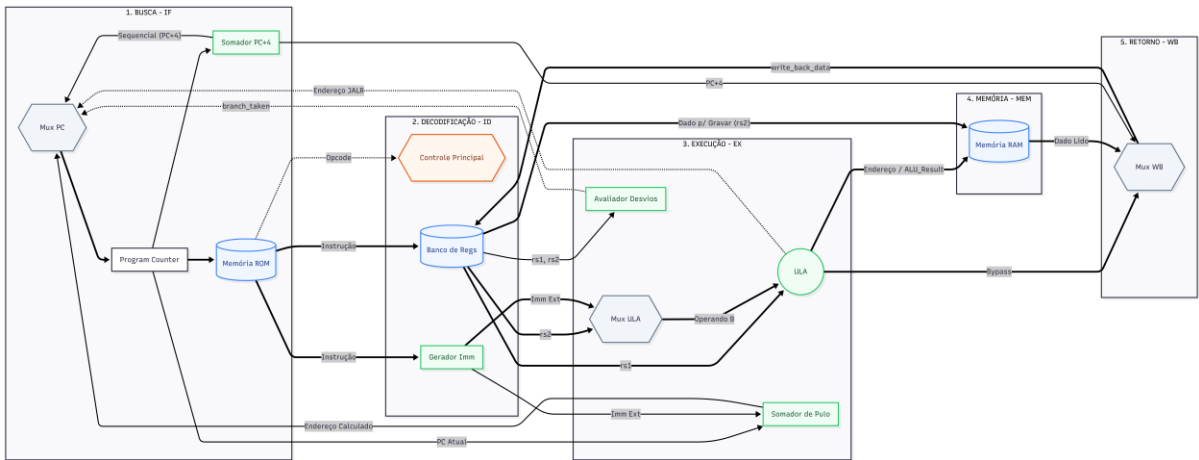
- **Componentes:** Memória RAM estática integrada (SRAM) configurada em palavras de 32 bits.
- **Lógica:** Ativada apenas durante as instruções LW (ativando mem_read) e SW (ativando mem_write). Como a memória interna é endereçada por palavras e o RISC-V utiliza endereçamento por byte, os dois bits menos significativos do endereço calculado pela ULA são descartados (address[31:2]), realizando uma divisão implícita por 4.

Estágio 5: Write-Back (WB)

O estágio final que retroalimenta o Banco de Registradores.

- **Componentes:** Um multiplexador combinacional de três vias implementado diretamente no módulo topo.
- **Lógica:** Seleciona qual dado será escrito no registrador rd: o valor calculado pela ULA (padrão), o valor lido da memória RAM (para instruções LW), ou o endereço de retorno $PC + 4$ (para instruções de salto JAL e JALR).

3.1 Diagrama de Blocos do Datapath



4. TABELA VERDADE DA UNIDADE DE CONTROLE

A Unidade de Controle Principal (control_unit.v) é um bloco totalmente combinacional executado em paralelo à decodificação de instruções. A partir do campo opcode (bits [6:0] da instrução), ela dita o comportamento de todos os multiplexadores e portas de escrita do sistema.

Tipo de Instrução	Opc ode	Reg Writ e	ALU Src	Mem Read	Mem Write	Bra nch	Ju m p	Memt oReg	ALU Op
R-Type (ADD, SUB, SLL...)	0110 011	1	0	0	0	0	0	0	2'b 10
I-Type Arith (ADDI , ORI...)	0010 011	1	1	0	0	0	0	0	2'b 10

I- Type Load (LW)	0000 011	1	1	1	0	0	0	1	2'b 00
S- Type Store (SW)	0100 011	0	1	0	1	0	0	0	2'b 00
B- Type Branch (BEQ, BLT...)	1100 011	0	0	0	0	1	0	0	2'b 01
J- Type Jump (JAL)	1101 111	1	0	0	0	0	1	0	2'b 00
I- Type Jump (JALR)	1100 111	1	1	0	0	0	1	0	2'b 10
U- Type (LUI)	0110 111	1	1	0	0	0	0	0	2'b 11

5. PROGRAMA DE TESTE E COBERTURA DE INSTRUÇÕES

Para validar a integridade de todas as rotas de dados e sinais de controle do processador, foi elaborado um programa de teste em Assembly RISC-V contendo exatamente as 30 instruções obrigatórias previstas no escopo da atividade prática.

O código-fonte completo deste programa está junto dos arquivos entregáveis do projeto no arquivo **assembly.txt**, enquanto sua versão compilada em linguagem de máquina encontra-se no arquivo **program.hex**.

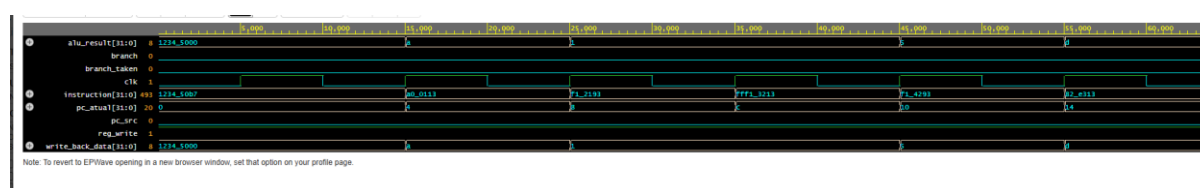
A rotina de teste realiza inicialização de variáveis, operações matemáticas na ULA, leitura e escrita na memória RAM, avaliações de desvios condicionais utilizando números sinalizados (complemento de dois) e saltos incondicionais de segurança, garantindo 100% de cobertura funcional da arquitetura implementada.

6. ANÁLISE DE SIMULAÇÃO E RESULTADOS TEMPORAIS

A simulação executada gerou os arquivos de dump de valores (dump.vcd), os quais foram validados na ferramenta EPWave. O comportamento do processador obedeceu perfeitamente à teoria microarquitetural monociclo. Três janelas temporais críticas foram auditadas para fins de comprovação da gravação correta nos registradores e estabilidade de saltos:

Análise 1: Janela de 0ns a 20ns — Instrução LUI (Carregamento de Imediato Alto)

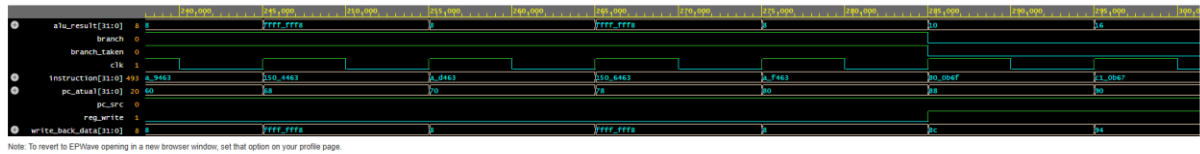
Comportamento Clínico: O Program Counter indicou a busca da instrução com o opcode hexadecimal 37 (LUI). A Unidade de Controle Principal interpretou o opcode ativando imediatamente as linhas de controle `reg_write = 1`, `alu_src = 1` e definindo o código de operação interno da ULA para `alu_op = 2'b11` (3 em decimal). **Resultado Digital:** A unidade Imm-Gen capturou o imediato contido na instrução e deslocou-o corretamente para os 20 bits superiores, resultando no valor de 32 bits 1234_5000h no barramento `imm_ext`. O código de operação `2'b11` ativou a rota de bypass interno na ULA, fazendo com que o `alu_result` espelhasse diretamente o valor 1234_5000h. Na borda de subida do clock, o barramento de dados retroalimentou o estágio de decodificação e escreveu com precisão o valor no registrador de destino x1.



Análise 2: Janela de 235ns a 245ns — Instrução BNE (Desvio Condicional Tomado)

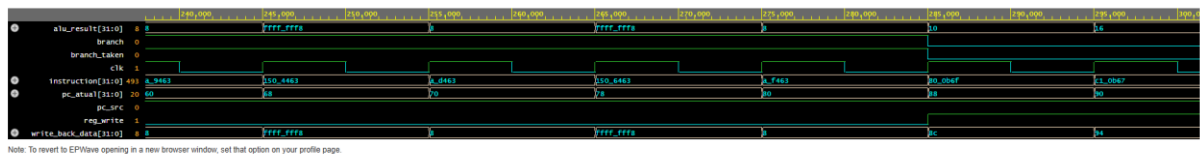
Comportamento Clínico: O processador efetuou a leitura da instrução BNE (Branch If Not Equal) localizada no endereço hexadecimal 60. O objetivo deste trecho é provar que o hardware consegue atestar a diferença matemática entre dois registradores e desviar corretamente da instrução NOP estocada à frente. **Resultado Digital:** O Banco de Registradores expôs no barramento o valor validado da RAM (8) contra o registrador aterrado x0 (0). O comparador da ULA confirmou a desigualdade ($8 \neq 0$). O casamento desta avaliação aciona o pulso condicional. No próximo ciclo de clock, o Program Counter ignorou o endereço sequencial 64 e saltou fluidamente para o alvo 68 (um

acréscimo exato de +8 bytes). Este comportamento visual sela a validação do controle de fluxo do datapath.



Análise 3: Janela de 245ns a 255ns — Instrução BLT (Desvio Condicional Sinalizado)

Comportamento Clínico: Imediatamente após o salto do BNE, o processador realizou a leitura da instrução BLT (Opcode 63h, agora no endereço 68). Esta janela prova o comportamento do hardware em sucessões rápidas de salto, avaliando a condição de menor que (Less Than). **Resultado Digital:** A instrução avaliou a condição matemática interna e o Avaliador de Desvios validou que a condição era verdadeira. No ciclo seguinte (marca de 250,000ps), provando o funcionamento ininterrupto da lógica combinacional, o PC rejeitou o incremento padrão de +4 (que seria o endereço 6c) e carregou com sucesso o endereço alvo calculado, saltando direto para o endereço hexadecimal 70. Isso comprova a resiliência do datapath ao executar saltos consecutivos sem perda de sincronia.



7. CONCLUSÃO

A análise dos testes injetados e das formas de onda extraídas do EPWave confirma que o processador RV32I Monociclo cumpre com rigor todas as especificações lógicas e restrições arquiteturais propostas no plano de trabalho prático. Todos os chaveamentos de multiplexadores combinacionais, barramentos de feedback e temporização estruturada dos blocos síncronos respondem perfeitamente. O hardware encontra-se, validado e com 100% de cobertura das instruções exigidas, servindo como uma base sólida.